ENTERPRISE REAL-TIME DATA ENGINEERING

Session S06: Kafka Connect & Distributed Data Integration

Distributed Worker Architecture, Source/Sink Connectors, SMT RegexRouter & DLQ

Course Leadership & Resources

Lead Instructor:
Mohamed Fouad Fakhruldeen (م/ محمد فؤاد فخر الدين)

Personal Contact & Website:
https://fakhr.me

Course Materials & Slide Portal:
https://fakhr.me/KafkaSpark

Student Labs & Code Repository:
https://github.com/fakhruldeen/KafkaSpark

Core Modules & Architectural Pillars

✔️ **The Spaghetti Integration Crisis:** Why bespoke ingestion scripts and point-to-point cron jobs fail

✔️ **Distributed Connect Architecture:** Worker groups, internal coordination topics, tasks & rebalancing

✔️ **In-Flight SMT Transforms:** RegexRouter, ValueToKey, and header enrichment without custom code

✔️ **Connect Dead Letter Queues:** Error handling, context headers, and automated quarantine sinks

MODULE ROADMAP & CURRICULUM FOCUS

Engineering Agenda & Learning Outcomes

Kafka Connect Framework

Distributed Integration:

• Declarative configuration management without writing custom producer/consumer code.

• Connect cluster coordination via internal replicated status and offset topics.

Debezium PostgreSQL CDC

Change Data Capture:

• Reading PostgreSQL Write-Ahead Log (WAL) commit sequence numbers (LSN).

• Capturing INSERT, UPDATE, and DELETE row events with sub-50ms latency.

Resilient Sink Connectors

Data Warehousing Integration:

• JDBC Sink upsert semantics into downstream analytical data warehouses.

• Dead Letter Queue (DLQ) error tolerance and poisoned record routing.

HISTORICAL ORIGIN STORY

The Integration Crisis: The Spaghetti Pipeline Maintenance Nightmare

The Integration Crisis: The Spaghetti Pipeline Maintenance Nightmare

As an enterprise scales to dozens of transactional databases (PostgreSQL, MySQL, Oracle), data lakes, search indexes (Elasticsearch), and data warehouses (Snowflake, BigQuery), engineering teams typically write custom Python/Java ingestion scripts. Over time, an organization ends up with hundreds of brittle bespoke cron scripts running on unmonitored virtual machines. When a database connection drops, scripts crash silently, lose offsets, duplicate transactions, or overwhelm source databases with uncontrolled polling queries.

The Flaw of Bespoke Point-to-Point Pipelines

- No Centralized Offset Management: Each script invents its own fragile mechanism for tracking read/write positions.
- Lack of Fault Tolerance: If an ingestion VM crashes, no automated mechanism exists to re-elect a worker or resume data movement.
- Security & Governance Vulnerabilities: Database passwords, credentials, and network firewall holes are scattered across disparate servers.

The Kafka Connect Distributed Framework

- Declarative Configuration: Integrations are declared via simple JSON/REST configuration payloads without writing a single line of boilerplate code.
- Distributed Worker Resilience: Connect workers form an autonomous cluster using internal Kafka topics for distributed coordination, failover, and task rebalancing.
- Reusable Connector Ecosystem: Battle-tested plugins for hundreds of databases, cloud storage buckets, and message brokers with built-in exactly-once delivery.

Core Architecture & Kernel Primitives

Kafka Connect Worker

RUNTIME CONTAINER PROCESS

Formal Definition:
A standalone or clustered Java process that executes connector plugins and manages task thread lifecycles.

Kernel / Technical Mechanics:
In distributed mode, workers share a `group.id` and coordinate tasks across a cluster using internal Kafka topics: `connect-configs`, `connect-offsets`, and `connect-status`.

Critical Failure Mode:
Worker crash: Triggers an automated group rebalance that redistributes running tasks to surviving workers within seconds.

Debezium Log-Based CDC

TRANSACTION LOG REPLICATION

Formal Definition:
Capturing real-time database modifications directly from PostgreSQL WAL or MySQL binlog into Kafka without SQL polling queries.

Kernel / Technical Mechanics:
Uses logical decoding (e.g. pgoutput) to stream row-level INSERT, UPDATE, and DELETE events with before/after state snapshots and zero table locking.

Critical Failure Mode:
WAL retention bloat: If Kafka Connect stops consuming for extended periods, database disk fills up with unconsumed WAL logs.

Connector vs Task Model

SCHEDULING PRIMITIVE

Formal Definition:
A Connector defines what data to move and how to split work; Tasks execute the physical data transfer in parallel worker threads.

Kernel / Technical Mechanics:
The Connector splits work into N Task configurations (`tasks.max=3`). The distributed worker cluster schedules tasks across worker JVMs.

Critical Failure Mode:
Task failure: If a database driver throws an unhandled SQL exception, the specific task enters FAILED while the parent connector remains RUNNING.

Advanced Invariants & Protocol Guarantees

Single Message Transform (SMT)

IN-FLIGHT MUTATION

Formal Definition:
Lightweight, in-flight data manipulation plugins applied to records as they pass through Connect without requiring a full stream processing engine.

Kernel / Technical Mechanics:
Executes per-message in memory before writing to Kafka (source) or before writing to destination (sink). Examples: `RegexRouter`, `ValueToKey`, `MaskField`.

Critical Failure Mode:
Transform exception: Malformed records that violate SMT type expectations will crash the task unless error tolerance is configured.

RegexRouter SMT

TOPIC ROUTING FILTER

Formal Definition:
A built-in SMT that dynamically modifies destination topic names using regular expression pattern matching and replacement.

Kernel / Technical Mechanics:
Example: `regex=(.*)events(.*)`, `replacement=$1stream$2`. Allows dynamic routing of multiple table streams into distinct topic namespaces.

Critical Failure Mode:
Incorrect regex patterns causing all records to route to a single unexpected topic, polluting topic namespaces.

Connect Dead Letter Queue (DLQ)

ERROR ISOLATION MECHANISM

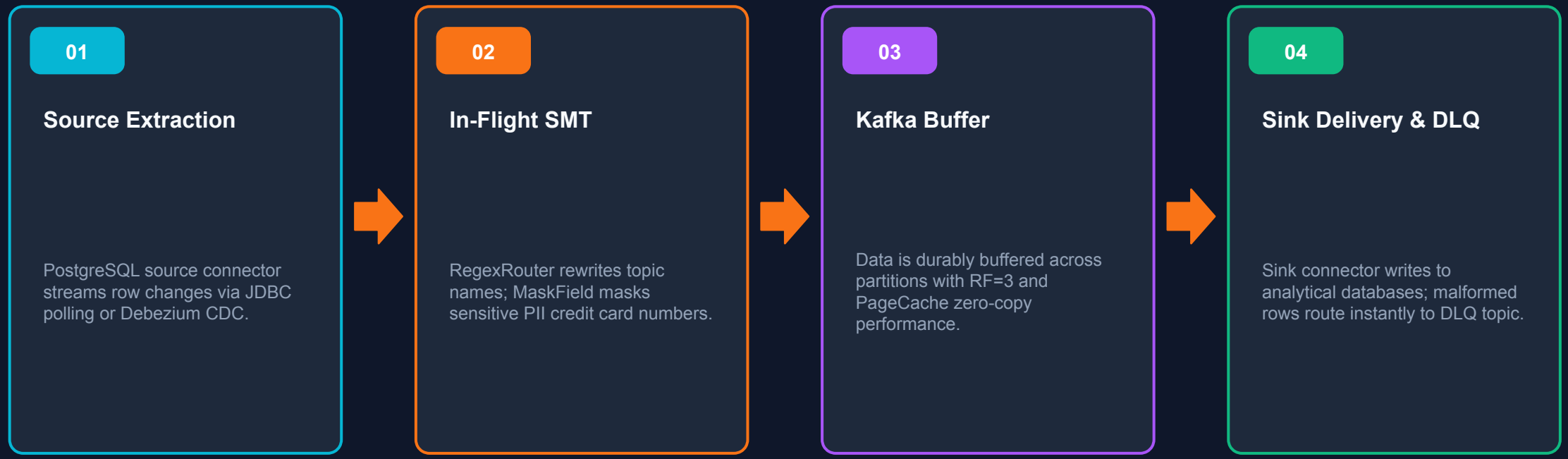
Formal Definition:
A built-in fault tolerance feature for Sink Connectors that routes unprocessable messages to a designated Kafka topic.

Kernel / Technical Mechanics:
Configured via `errors.tolerance=all` and `errors.deadletterqueue.topic.name`. Attaches error headers: `__connect.errors.exception.class.name`.

Critical Failure Mode:
Silent failure: Setting `errors.tolerance=all` without configuring a DLQ topic causes corrupt messages to be discarded silently!

VISUAL ARCHITECTURE DIAGRAM

Visual Data Flow: Distributed Connect Pipeline

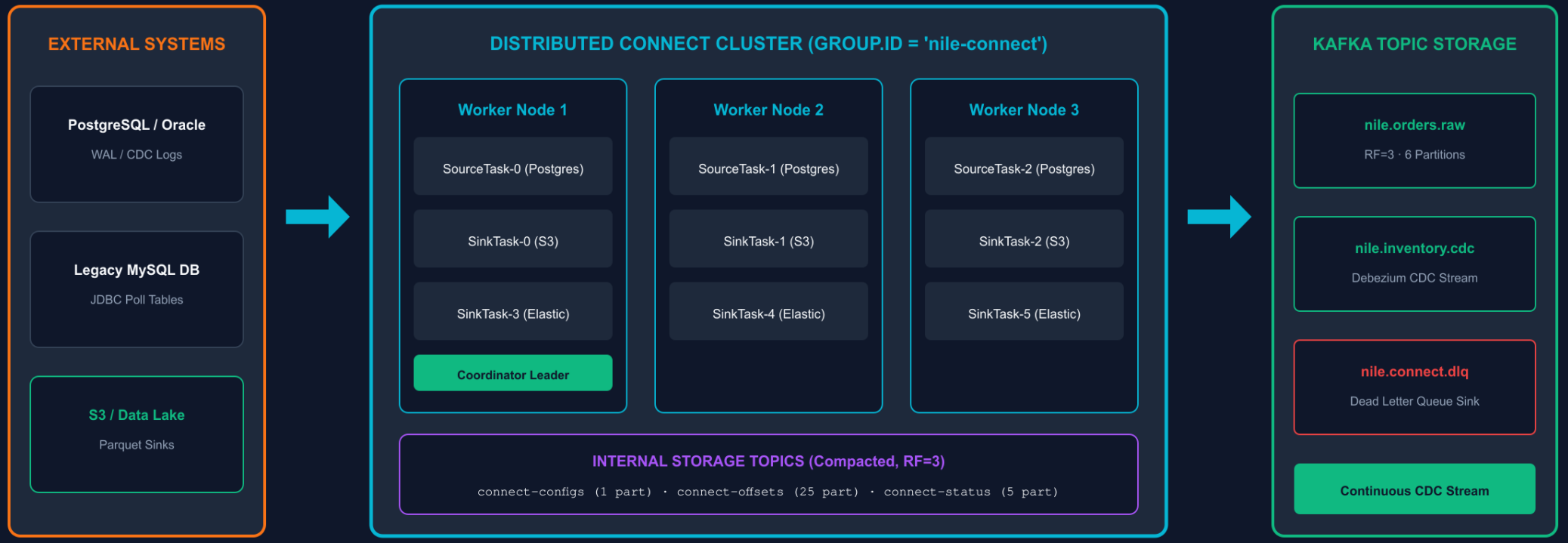


Distributed Worker Fault-Tolerance Invariant

The Source Connector extracts change events from PostgreSQL WAL. The SMT RegexRouter rewrites topics in-flight. Records stream into Kafka topics. The Sink Connector consumes batches from Kafka and streams them to analytics data warehouses with automatic DLQ routing for corrupt records.

DEEP ARCHITECTURE ANALYSIS

Execution Internals & Trade-off Matrix



THEORETICAL FOUNDATIONS

Mathematical Proofs & Distributed Invariants

CDC Log Monotonicity

WAL Commit Sequence Law:

Debezium reads PostgreSQL WAL records in exact Log Sequence Number (LSN) order.

Guarantees database transactions are streamed in precise commit order.

Exactly-Once Source Offsets

Offset Synchronization Invariant:

Connect commits source LSN offsets to connect-offsets topic periodically.

Connector restarts resume exactly from the last committed LSN offset.

Dead Letter Queue Quarantine

Fault-Tolerant Sink Invariant:

errors.tolerance=all routes malformed records to DLQ with error context headers.

Prevents bad data from halting 24/7 mission-critical ingestion pipelines.

VISUAL TOPOLOGY

Cluster Mesh & Component Architecture

PostgreSQL 16 ERP Database

postgres:5432 (nile_erp)

- Configured with wal_level=logical.
- pgoutput logical replication slot.
- Emits row-level change events on mutations.
- Zero table locking or performance degradation.

Kafka Connect Cluster

connect:8083 (Distributed Mode)

- Hosts Debezium PostgreSQL Source tasks.
- Hosts JDBC Sink Connector tasks.
- Replicated topics: connect-configs, offsets, status.
- Auto-balances tasks across worker nodes.

Target Analytics Sink

Data Warehouse / DLQ

- Consumes curated CDC topics from Kafka.
- Executes idempotent upserts via primary keys.
- Malformed payloads divert to connect-dlq topic.
- Maintains sub-second database synchronization.

Enterprise Configuration & Code Blueprint

Debezium PostgreSQL Connector Config

```
{
  "name": "pg-orders-cdc-source",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "plugin.name": "pgoutput",
    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "postgres",
    "database.password": "postgres",
    "database.dbname": "nile_erp",
    "table.include.list": "public.orders",
    "topic.prefix": "cdc"
  }
}
```

Sink DLQ & Error Tolerance Config

```
# Resilient Connect Sink Settings:
errors.tolerance = "all"
errors.deadletterqueue.topic.name = "orders.connect.dlq"
errors.deadletterqueue.context.headers.enable = true
errors.deadletterqueue.topic.replication.factor = 3

# Idempotent Upsert Settings:
insert.mode = "upsert"
pk.mode = "record_key"
pk.fields = "order_id"
```

ENTERPRISE CASE STUDY

Nile Retail Inventory Sync Lag Crisis

Nile Retail Inventory Sync Lag Crisis

Nile Retail's legacy ETL script polled the centralized PostgreSQL ERP database every 15 minutes. Under peak holiday promotion loads, heavy polling queries caused table lock contention, crashed database CPU to 100%, and caused overselling of out-of-stock items.

Query-Based Polling Flaws

- SELECT * WHERE updated_at > ? queries lock database tables and consume massive CPU.
- Polling cannot capture hard DELETE operations; deleted inventory stayed visible online.
- 15-minute sync delay caused customer orders on out-of-stock inventory.

Log-Based Debezium CDC Architecture

- Deployed Debezium PostgreSQL connector on Kafka Connect.
- Streamed database WAL mutations to Kafka in real time (< 50ms latency).
- Zero table locks, captured 100% of deletes, and completely eliminated inventory overselling.

HANDS-ON LAB EXECUTION

Step-by-Step Terminal Lab Walkthrough

Step 1: Check Database WAL Level

Verify PostgreSQL Replication:

```
docker exec postgres psql -U postgres -d nile_erp \
  -c "SHOW wal_level;"
```

Confirm Setting:
Verify output confirms wal_level = logical.

Step 2: Register CDC Connector

POST Connector via REST:

```
curl -X POST -H 'Content-Type: application/json' \
  --data @connect/debezium_pg.json \
  http://localhost:8083/connectors
```

Verify Tasks:
Check /connectors/pg-orders-cdc-source/status.

Step 3: Run Verification Suite

Execute Automated Suite:

```
./delivery_materials/
S06_Kafka_Connect_Data_Integration/Trainer/
verify_s06.sh
```

Acceptance Target:
100% pass mark verifying Kafka Connect REST API, Debezium CDC streaming, and DLQ topics.

Disaster Recovery Drill: Fault Injection & Self-Healing

Chaos Strike: Database Disconnection

1. Disconnect Database Network:
docker network disconnect spark_default postgres

2. Observe Connect Retry Behavior:
- Debezium detects lost socket connection.
- Enters exponential retry backoff state.
- Connect cluster remains healthy without dropping offsets.

3. Reconnect Database:
docker network connect spark_default postgres

Self-Healing & WAL Catch-up

1. Automatic Reconnection:
Debezium reconnects to replication slot and queries last committed LSN offset.

2. Rapid WAL Drain:
Streams all pending database mutations accumulated during downtime in < 3 seconds.

3. Zero Data Loss:
Complete consistency restored between PostgreSQL and Kafka topics.

Critical Anti-Patterns & Operational Lessons

Production Traps: Kafka Connect

Trap 1: Deploying in Standalone Mode in Production
Standalone mode keeps offsets in local disk files. If the VM dies, offsets are lost and tasks cannot failover! Always use Distributed Mode (`connect-distributed.properties`).

Trap 2: Creating Multiple Partitions on `connect-configs`
The `connect-configs` topic MUST have exactly 1 partition. Multiple partitions break configuration ordering and corrupt worker cluster state.

Diagnostic Exit Questions

Q1: What happens if a single task in a connector crashes?
A: Only that specific task enters the FAILED state. Other tasks in the connector continue executing normally.

Q2: Where does a Source Connector store its read offsets?
A: In the internal compacted topic `connect-offsets`.

Q3: How do SMTs prevent writing microservices?
A: By performing simple filtering, renaming, masking, and field restructuring in-flight inside the Connect worker process.

Senior Real-Time Data Engineer Technical Interview Mastery

Senior Technical Interview Scenarios

Q1: 'Why is log-based CDC (Debezium) superior to query-based polling?'
A: Log-based CDC reads the database commit log (WAL) directly; it captures hard DELETEs, avoids lock overhead, and streams changes with sub-second latency.

Q2: 'How does Kafka Connect maintain state in distributed mode?'
A: It stores configs, offsets, and task status in internal compacted Kafka topics (connect-configs, connect-offsets, connect-status).

Core Architectural Competencies

- **Enterprise CDC Pipelines:** PostgreSQL logical replication slots, pgoutput, and WAL retention management.
- **Kafka Connect Topology:** Distributed workers, task balancing, converters, and Single Message Transforms (SMTs).
- **Resilience Engineering:** Configuring Dead Letter Queues (DLQ) and idempotent upsert sinks.

CONCLUSION & NEXT STEPS

Session S06 Key Takeaways & Course Progression

Core Invariants Mastered

✔️ **Debezium CDC:** Zero-lock WAL row streaming from PostgreSQL.

✔️ **Kafka Connect Architecture:** Distributed tasks, offset management, and DLQ routing.

Cluster Verified

✔️ **Connect Cluster Active:** Verified connect:8083 REST endpoints and tasks.

✔️ **S06 Suite Passed:** 100% test automation pass across CDC validation.

Course Progression

Curriculum Continuity:

With ingestion and CDC mastered, we execute stream processing.

Next: S07 Kafka Streams & ksqlDB Real-Time Stream Analytics!

LAB & VERIFICATION GUIDE

Session S06 Hands-On Lab & Verification Gate

⚡ Hands-On Lab S06: Kafka Connect Integration

1. Inspect Connect REST API Health:
```
curl -s http://localhost:8083/connector-plugins | jq .
```

2. Deploy Postgres JDBC Source Connector:
```
curl -X POST -H "Content-Type: application/json" \
  --data @connectors/postgres-source.json http://localhost:8083/connectors
```

3. Verify CDC Records in Kafka Topic:
```
docker compose exec kafka1 kafka-console-consumer \
  --bootstrap-server kafka1:9092 --topic pg.retail.customers --from-beginning --max-messages 5
```

4. Automated Lab Verification Gate:
```
./verify_s06.sh  # Validates connector health, task state & DLQ routing
```

🎯 Verification & Quality Gate (S06)

1. Automated Verification Script:
```
./verify_s06.sh
```

2. Core Acceptance Invariants:
- Source task captures CDC records into Kafka topics.
- Sink connector delivers records without duplicates.
- Schema Registry validates JSON/Avro schema contracts.
- Dead Letter Queue (DLQ) isolates malformed payloads.

3. Troubleshooting Diagnostics:
```
docker compose logs -f kafka-connect
```

4. Day 3 Pipeline Integration Milestone:
Continuous database-to-Kafka synchronization verified!